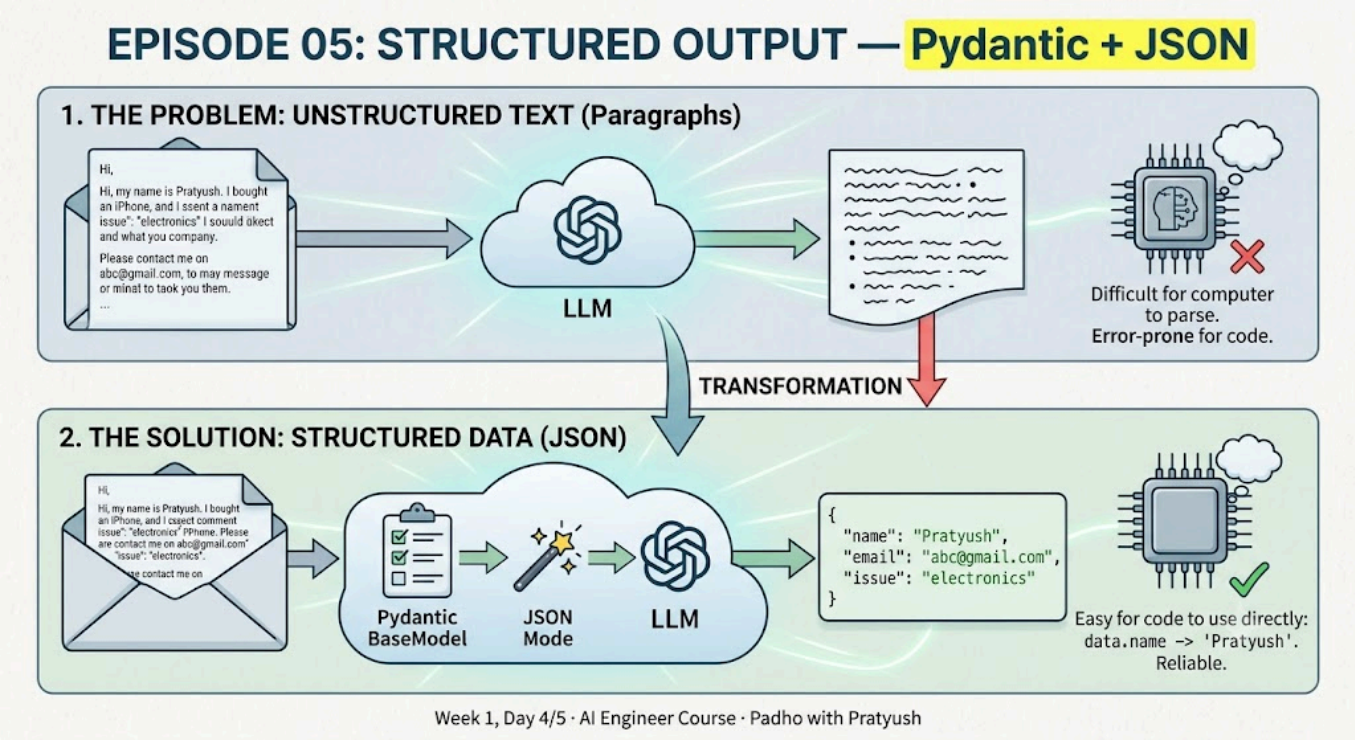
Day 5

Episode 05 — Structured Output: Pydantic + JSON — Detailed Notes

AI Engineer Course (8 Weeks) · Padho with Pratyush · Week 1, Day 4/5

The core idea



This episode is entirely about **structured output** — making an LLM return clean, machine-readable data instead of a free-form paragraph. Two tools solve it: **Pydantic** (define the exact shape of the data you want) and **JSON mode** (force the LLM to actually return that shape). By the end you can build a mini-project that extracts specific fields from messy text.

1. Why raw LLM text is a problem

- So far you sent the LLM a **prompt (a string)** and read back a **response (also a string / paragraph)** via `response.choices[...]`.
- A paragraph is **great for humans** — easy to read and understand.
- It is **terrible for computers**. Reading a paragraph and pulling out specific values (“parsing”) is one of the hardest things a program can do.
- In a real company, an LLM’s output rarely goes to a human. It goes to **another program** (C++, Java, JavaScript, Go, C, etc.) that must make a decision from it.

- These agents run **millions of times a day** — no human can read a million responses. So the output *must* be structured for the next program.

Key takeaway: String parsing/extraction is *"one of the hardest things to do in a computer program as of today."* This gap is why ~80% of AI app development is harder than it should be.

2. The motivating example — a customer complaint email

A customer sends a messy complaint:

"Hi, my name is Pratyush. I bought an iPhone from your store and it stopped working. Please contact me on abc@gmail.com. Regards, Pratyush, 82155…"

Your LLM's job: **extract the fields** → `name`, `email`, `phone`, `issue` (issue = a category like *electronics*).

- **A human** reads it instantly and picks out the values.
- **An LLM** often adds fluff: *"Sure, let me help you with that. Name is Pratyush. Phone is 8215…"* — still a **string**, still unstructured.
- The next program then has to re-parse this — find "name", skip the word "is", grab the value, ignore the greeting, etc. Painful and error-prone. Around 95% of learners can't reliably write this string parsing.

Conclusion: You can't pass a raw string when the consumer is another program. You need **structured output** with *exactly* those fields — no more, no less.

3. JSON — the solution format

- **JSON = JavaScript Object Notation.** It existed long before LLMs — built so computers/servers could share data reliably (servers in the US, Mumbai, Delhi, etc. all need a common format).
- Sending data as a **pure string** causes problems; JSON is a **widely-used, popular communication format** because it's easy for both humans *and* computers to read.

What a JSON object looks like (curly braces, key : value pairs):

```
{
  "name": "Pratyush",
  "email": "abc@gmail.com",
  "phone": 82155,
  "issue": "electronics"
}
```

Why JSON beats a string:

- No manual line-by-line parsing needed.
- JSON **libraries** already exist. The consumer just writes `data.name` → gets `"Pratyush"` directly; `data.phone` → gets the number directly.
- One line of code vs. parsing a 5–6 line paragraph.

Structured vs. unstructured — the watchman analogy:

- A watchman told to log each visitor's name + number plate can either:
- Write a **paragraph**: *"Pratyush came at 2 o'clock in a DL-xxx car"* → **unstructured** (only he understands it), or
- Note `name: Pratyush`, `number_plate: DL-XE-7338` → **structured** (easy for anyone to read).
- Structured is easier to **write** and easier to **read**.

Plan: Always convert the LLM's output into JSON, store it, *then* pass it forward — never pass the raw string.

4. Pydantic — defining what you want

Pydantic is a Python library that lets you tell your program *exactly* which fields you need and their types. Whatever story the LLM tells, the **final response fills only these fields**.

a) Import BaseModel

```
from pydantic import BaseModel
```

`BaseModel` means: "this is my base — I only want these fields; ignore everything else (father's name, Aadhaar number, address, etc.)."

b) Create a class (e.g., `Ticket`) — treat each complaint email as a "ticket." List the fields you need + their data types:

```
class Ticket(BaseModel):
    name: str
    email: str
    issue: str   # category of the issue
```

⚠️ **Most common mistake:** forgetting `(BaseModel)`. Without it you get *"Ticket has no attribute model_json_schema"*. The class **must** inherit `BaseModel`.

c) Build a schema from the class. A schema is the *framework* describing the required shape:

```
schema = Ticket.model_json_schema()
```

Don’t memorize the syntax — it sticks after writing 3–4 programs.

5. Wiring it into the LLM call

a) Response format — tell the API you want a JSON object back:

```
response_format = {"type": "json_object"}
```

b) System prompt — describe the LLM’s behaviour (an f-string that injects the schema):

```
system_prompt = f"""Extract the information and return it strictly as a JSON
output
matching this schema. Do not make up your own schema: {schema}"""
```

c) System message:

```
message_system = {"role": "system", "content": system_prompt}
```

d) User message (the actual complaint):

```
prompt = f"""This is a customer ticket. Please extract the personal
information from this: {text}"""
message = {"role": "user", "content": prompt}
```

e) Messages array — system first, then user:

```
messages = [message_system, message]
```

f) Pass `response_format` to the completion call:

```
response_format=response_format
```

Two errors demonstrated (and their fixes)

1. `Ticket has no attribute model_json_schema` → you forgot to inherit `BaseModel`. Add `(BaseModel)` to the class.
2. `Messages must contain the word 'json' in some form to use response_format` → the word **“json” must appear** in your system or user prompt. `response_format` sets the

shape; the prompt must explicitly ask for JSON. Fix: add *"give me a JSON output"* to the system prompt.

6. Reading the JSON output back

Once it works, the LLM returns a clean JSON object with only `name`, `email`, `issue`. Anything not in the schema (address, girlfriend's name, etc.) is **ignored** — because it's not in your `Ticket` /schema.

Parse it in 3 lines:

```
import json

raw_json = answer                  # the raw JSON string from the LLM
data_file = json.loads(raw_json)   # parse string -> Python dict
ticket = Ticket(**data_file)       # unpack dict -> Ticket object

print(ticket.name)
print(ticket.email)
print(ticket.issue)
```

⚠️ **Mistake shown:** writing `data` when the variable is actually `data_file` → *"data is not defined"*. Match variable names exactly.

Result prints cleanly (e.g., `Pratyush`, the email, `iPhone not working`) — each field ready to pass to the next program.

7. Extra points mentioned

- **Temperature:** raising temperature makes output more varied/messy (the video shows stray `*` characters appearing around fields). For extraction tasks, keep it **low (temperature = 0)** so output is deterministic and clean.
- **Literal types:** mentioned as a way to constrain the LLM's output to a fixed set of allowed values (e.g., issue categories) — use `Literal[...]`.
- **Setup recap:** `uv init day4` → `cd day4` → `uv venv --python 3.11` → activate venv → `uv add groq python-dotenv pydantic` → run with `python json_pydantic.py`. You reuse the Day 1 "hello LLM" code — this is iterative (copy-paste-and-extend), not rewriting from scratch.

8. Homework — Resume Parser mini-project

Build a program that:

1. Takes a **resume** (PDF or Word) and reads it via **file handling**.

2. Takes an **HR requirements list** — e.g., experience required (2 years), skills (`Python` , `C++` , `Java` , `JavaScript`), a required project type (e.g., a machine-learning project).
3. **Extracts** the relevant fields from the resume (experience, skills, projects) using the Pydantic + JSON pattern.
4. **Matches** resume against the HR list and outputs a **match percentage** (e.g., “this candidate is a 70% match”).

The same extraction pattern powers customer-support bots, resume parsers, and data pipelines — the skill that separates people who *play* with ChatGPT from people who *build* AI products.

One-line summary

LLMs return messy strings; downstream code needs clean data. **Pydantic defines the exact schema, JSON mode forces the LLM to return it, and** `json.loads` **+** `Model(**data)` **parses it back** — with `temperature=0` keeping extraction reliable.

My Code

```
import json

from groq import Groq

from dotenv import load_dotenv

import os

from pydantic import BaseModel



load_dotenv()

my_api=os.getenv("GROQ_API_KEY")

client=Groq(api_key=my_api)



if not my_api:

raise TypeError("API not found")



model="llama-3.3-70b-versatile"
```

```
# text = """Hello, my name is Pratyush. I purchased an iPhone which stopped
working.

# My address is Delhi. Please contact me on abc@gmail.com. Regards,
Pratyush."""



# prompt=f"""This is a custom ticket, please extract information from this
{text}"""



# messages=[{

# "role":"system",

# "content":prompt

# },

# {

# "role":"user",

# "content":text

# }]



# response=client.chat.completions.create(

# model=model,

# messages=messages

# )



# print(response.choices[0].message.content)
```

```
text = """Hello, my name is Pratyush. I purchased an iPhone which stopped
working.

# My address is Delhi. Please contact me on abc@gmail.com. Regards,
Pratyush."""



class Ticket(BaseModel):

name:str

email:str

issue:str



schema = Ticket.model_json_schema()



response_format={

"type":"json_object"

}



user_prompt=f"This is a custom ticket, please extract information from this
{text}"



system_prompt=f"Extract info from {text} but only return based in json type
{schema}"

message_system={

"role":"system",

"content":system_prompt
```

```
}



message_user={

"role":"user",

"content":user_prompt

}

messages=[message_system,message_user]



response=client.chat.completions.create(model=model, messages=messages,
response_format=response_format)

answer=response.choices[0].message.content

print(response.choices[0].message.content)




# how to send to next chat model

raw_json=answer

data_file=json.loads(raw_json)

ticket=Ticket(**data_file)

print(ticket.name)

print(ticket.email)

print(ticket.issue)
```